

Lab Test Subjects

GPIO

Binary Counter Using the four LEDs on the lab board (`LED_RED`, `LED_GREEN`, `LED_BLUE`, `LED_YELLOW`), implement a 4-bit binary counter.

- Assign each LED to represent a bit from the Least Significant Bit (LSB) to the Most Significant Bit (MSB).
 - The counter should start at 0 (all LEDs OFF) and update its value every **500 milliseconds**.
 - **The Switch Action:** Use switch `S1` to control the counting direction.
 - If `S1` is **released**, the counter counts **UP** (incrementing by 1). When it reaches 15 (all LEDs ON), it must wrap around back to 0.
 - If `S2` is **pressed**, the counter counts **DOWN** (decrementing by 1). When it reaches 0 (all LEDs OFF), it must wrap around to 15.
-

The Combination Lock Implement a digital combination lock using switches `S1` and `S2`, and `LED_RED` and `LED_GREEN`.

- Initially, set `LED_RED` to ON and `LED_GREEN` to OFF to indicate a "locked" state.
 - The unlock sequence is a specific pattern: `S1`, `S1`, `S2`, `S1`.
 - If the user enters the correct sequence, turn `LED_RED` OFF and `LED_GREEN` ON for 3 seconds, then automatically lock it again.
 - If the user presses an incorrect button at any point in the sequence, rapidly flash `LED_RED` 3 times and reset the input tracker.
-

Long Press vs. Short Press Evaluator Write a program that differentiates between a quick tap and a long hold on switch `S1`. Measure the time between the falling edge (button pressed) and the rising edge (button released).

- If the button is held for less than 500 milliseconds (a short tap), toggle the state of `LED_GREEN`.
- If the button is held for 500 milliseconds or more (a long press), toggle the state of `LED_RED`.
- The LED states should only change *after* the button is released.

Software PWM (Breathing LED) Hardware timers usually handle Pulse Width Modulation (PWM), but you can simulate it in software using fast loops. Create a "breathing" effect on `LED_YELLOW` where it smoothly fades from completely off to completely on, and back to off, over a span of 2 seconds.

Simon Says Memory Game Create a memory game using the four buttons (`S1` to `S4`) and four LEDs (`LED_RED`, `LED_GREEN`, `LED_BLUE`, `LED_YELLOW`).

- Map each button to a specific LED (e.g., `S1` = Red, `S2` = Green, etc.).
 - The microcontroller plays a random sequence of LED flashes, starting with a sequence of one.
 - The user must repeat the sequence by pressing the corresponding buttons. Use `wait_for_falling_edge()` to read the user's inputs asynchronously.
 - If the user gets it right, add one more random flash to the sequence and play it again.
 - If the user makes a mistake, flash all four LEDs simultaneously three times to signal a "Game Over", then restart the game from sequence length one.
-

Tap Tempo Blinker Implement a feature where the user can dynamically set the blinking speed of `LED_YELLOW` by tapping a button.

- Continuously wait for the user to press `S1`.
 - Measure the time interval between consecutive presses using `Instant::now()` and `.duration_since()`.
 - Keep track of the last three tap intervals and calculate their average.
 - Run an asynchronous background loop that continuously blinks `LED_YELLOW` using that calculated average time as the delay. The blinking should update dynamically as the user continues to tap new tempos.
-

Inactivity Auto-Sleep Implement a power-saving simulation.

- When the system is "awake", `LED_GREEN` is solidly ON.
- If neither `S1` nor `S2` is pressed for a continuous duration of 5 seconds, the system enters "sleep mode". In sleep mode, `LED_GREEN` turns OFF and `LED_RED` turns ON.
- Any button press while awake should reset the 5-second countdown.
- Any button press while asleep should instantly wake the system up and restart the 5-second timer.

The "Hold to Charge" Progress Bar Simulate a charging mechanism that requires continuous user input.

- When **S1** is pressed and held, begin "charging".
 - After 1 second of continuous holding, turn on **LED_RED**. After 2 seconds, add **LED_GREEN**. After 3 seconds, add **LED_BLUE**. After 4 seconds, add **LED_YELLOW**.
 - If the user releases **S1** at any point *before* the 4 seconds are up, immediately turn off all LEDs and reset the progress.
 - If the user successfully holds it for the full 4 seconds, flash all 4 LEDs simultaneously three times to indicate "Fully Charged", then wait for the user to release the button before allowing another charge.
-

Strict Timed Passcode Implement a combination lock that requires a specific sequence (**S1**, **S1**, **S2**) to unlock (**LED_GREEN** turns on).

- Add a strict time constraint: The user has exactly 3 seconds to complete the entire sequence from the moment the *first* button is pressed.
 - If the 3-second timer expires before the sequence is finished, instantly flash **LED_RED** and reset the input tracker.
 - If an incorrect button is pressed at any point, instantly flash **LED_RED** and reset.
-

Logic Gate Simulator Turn your microcontroller into a hardware logic gate using two buttons (**S1**, **S2**) as inputs and **LED_BLUE** as the output.

- **AND Gate Mode:** **LED_BLUE** only turns ON if *both* **S1** and **S2** are actively being pressed at the same time.
- **OR Gate Mode:** **LED_BLUE** turns ON if *either* **S1** or **S2** (or both) are being pressed.
- **XOR Gate Mode:** **LED_BLUE** turns ON if **S1** is pressed OR **S2** is pressed, but *not* both.

The Manual Shift Register Create a system where an illuminated LED "shifts" left or right across the 4 LEDs (**LED_RED**, **LED_GREEN**, **LED_BLUE**, **LED_YELLOW**) based strictly on button presses.

- Initialize the system with only **LED_RED** turned on.
 - Every time **S1** is pressed, shift the light one position to the right. If it's at the far right, wrap around to the left.
 - Every time **S2** is pressed, shift the light one position to the left. If it's at the far left, wrap around to the right.
-

Hardware Demultiplexer A demultiplexer takes a single input signal and routes it to one of multiple outputs based on a "selector" switch.

- Use **S1** and **S2** as a 2-bit binary selector (e.g., neither pressed = **00**, only S2 pressed = **01**, only S1 pressed = **10**, both pressed = **11**).
 - Use **S3** as your "data" input.
 - Map the 2-bit selector to your 4 LEDs (00 = Red, 01 = Green, 10 = Blue, 11 = Yellow).
 - **The Behavior:** Whichever LED is currently selected by **S1** and **S2** should instantly mirror the exact state of **S3**. If **S3** is pressed, that specific LED turns on. If **S3** is released, it turns off. The other 3 unselected LEDs must remain perfectly off.
-

Any-Edge Interrupter Embassy provides a `wait_for_any_edge()` function. Use it to create a system that reacts instantly to both the press *and* the release of a button, without polling.

- Write a loop that awaits `wait_for_any_edge()` on **S1**.
 - When the await resolves, read the current state of the pin.
 - If the pin is low (pressed), turn **LED_BLUE** ON.
 - If the pin is high (released), turn **LED_BLUE** OFF.
 - Count and print how many total "edges" (state changes) have occurred since the board booted up.
-

Gray Code Cycler A standard binary counter changes multiple bits at once (e.g., **011** to **100** changes 3 bits). A "Gray Code" sequence ensures that only *one* bit (LED) changes state at a time, which prevents mechanical hardware glitches.

- Implement a 4-bit Gray code sequence on the 4 LEDs. The first few states are:
 - **0000** (All off)
 - **0001** (Yellow on)
 - **0011** (Blue + Yellow on)
 - **0010** (Blue on)
 - **0110** (Green + Blue on)
 - Use **S1** to advance to the next state in the sequence.
-

The Contextual "Shift" Key (GPIO) Treat **S4** like the "Shift" key on a keyboard, which modifies what the "Action" button (**S1**) does.

- Start with **LED_RED** and **LED_BLUE** OFF, and **LED_GREEN** and **LED_YELLOW** ON.
 - **Normal Mode (S4 Released):** * **LED_RED** perfectly mirrors **S1** (pressed = ON, released = OFF).
 - **LED_GREEN** perfectly inverses **S1** (pressed = OFF, released = ON).
 - **LED_BLUE** and **LED_YELLOW** must ignore **S1** and remain frozen in their default states.
 - **Shift Mode (S4 Held Down):**
 - The roles instantly swap to the other side of the board!
 - **LED_BLUE** now mirrors **S1**.
 - **LED_YELLOW** now inverses **S1**.
 - **LED_RED** and **LED_GREEN** must instantly freeze and ignore **S1** until **S4** is released.
-

The Manual Traffic Light (State Machine) Build a step-through traffic light controlled entirely by a single button, using a finite state machine (FSM).

- Use **S1** to advance to the next state. You must use `wait_for_falling_edge().await` so that holding the button doesn't rapidly cycle through the states.
 - **State 0:** **LED_RED** is ON.
 - **State 1:** **LED_RED** and **LED_YELLOW** are ON.
 - **State 2:** **LED_GREEN** is ON.
 - **State 3:** **LED_YELLOW** is ON.
-

The Directional Shift (Ping-Pong) Create a single illuminated LED that you can physically move left and right across the board.

- Start with **LED_RED** (the leftmost LED) ON.
- If **S2** is pressed (falling edge), shift the light one position to the right.
- If **S1** is pressed (falling edge), shift the light one position to the left.
- **The Boundary Rule:** If the light is at **LED_YELLOW** (far right) and **S2** is pressed, it should bounce back to **LED_BLUE**. If it is at **LED_RED** (far left) and **S1** is pressed, it should bounce back to **LED_GREEN**.

The Contextual Mode Switcher Build a system where the function of a button changes entirely depending on what "Mode" the system is in.

- **S1** is the "Mode" button. Pressing it cycles through 3 modes: Mode A, Mode B, and Mode C.
 - **LED_RED** indicates Mode A is active. **LED_GREEN** indicates Mode B. **LED_BLUE** indicates Mode C.
 - **S2** is the "Action" button.
 - In *Mode A*, pressing S2 toggles **LED_YELLOW** ON and OFF.
 - In *Mode B*, pressing S2 turns **LED_YELLOW** ON only while S2 is held down (flashlight mode).
 - In *Mode C*, pressing S2 immediately resets the system to Mode A and turns **LED_YELLOW** OFF.
-

The Hardware Comparator In digital electronics, a comparator takes two binary numbers and determines if one is greater than, less than, or equal to the other.

- Treat **S1** and **S2** as a 2-bit binary number (Group A). S1 is the high bit, S2 is the low bit.
 - Treat **S3** and **S4** as a second 2-bit binary number (Group B). S3 is the high bit, S4 is the low bit.
 - Write a continuous polling loop (`loop {}`) that continuously calculates the integer values of Group A (0 to 3) and Group B (0 to 3).
 - **Output:**
 - If Group A > Group B, turn ON **LED_RED**.
 - If Group A < Group B, turn ON **LED_BLUE**.
 - If Group A == Group B, turn ON **LED_GREEN**.
-

The "Press-and-Release" Sequence Validator Most sequence locks only care when a button is pressed. This one cares when it is released. You must track the entire lifecycle of the buttons to unlock the system.

- The user must follow this exact timeline of edges:
 1. Press and hold **S1** (S1 Fall).
 2. While holding S1, press and hold **S2** (S2 Fall).
 3. Release **S1** first (S1 Rise).
 4. Finally, release **S2** (S2 Rise).
- If this sequence is followed perfectly, turn ON **LED_GREEN**.
- If any edge happens out of order (e.g., they release S2 before S1), turn ON **LED_RED**. The system must now lock the user out until they completely "reset" by releasing all buttons.

PWM&ADC:

Digital Voltmeter Write a program that uses the potentiometer as a variable voltage source.

- Continuously read the raw ADC value from the potentiometer.
 - Calculate the actual real-world voltage (from `0.0V` to `3.3V`) using floating-point math (or millivolts using integer math for better performance).
 - Print the calculated voltage to the console every 500ms using `defmt::info!`.
-

Light Threshold Alarm Create a simple dark-detecting security system.

- Read the value of the photoresistor.
 - Determine a raw ADC threshold value that represents "darkness" in your room (you may need to print the values first to find a good number).
 - If the light level falls below this threshold, turn on the lab board's buzzer and `LED_RED`. If it is above the threshold, keep them off.
-

Software Low-Pass Filter Analog sensors are notoriously noisy, which can cause erratic behavior if used directly.

- Read the potentiometer value every 10ms.
 - Implement an Exponential Moving Average (EMA) filter: `filtered_value = (alpha * current_reading) + ((1.0 - alpha) * previous_filtered_value)`. Use an `alpha` of `0.1`.
 - Print both the `current_reading` and the `filtered_value` to the console. You should notice the filtered value changes much more smoothly when you turn the knob!
-

RGB Color Mixer

- Start with the RGB LED turned off.
 - **S1** increases the intensity of the **Red** channel by 10% each press (wrapping to 0% after 100%).
 - **S2** increases the intensity of the **Green** channel by 10% each press.
 - **S3** increases the intensity of the **Blue** channel by 10% each press.
-

Smooth Breathing LED Instead of stepping the brightness in chunks, create a continuous, smooth "breathing" effect on the **LED_BLUE** (or the blue channel of the RGB LED).

- The LED should smoothly transition from 0% to 100% intensity, and back down to 0%, over a period of 3 seconds.
 - Use a fast **loop** with small microsecond delays and tiny duty cycle increments to create the illusion of a perfectly smooth fade.
-

The "Smart" Nightlight Build a nightlight that automatically compensates for the ambient light in the room.

- Read the ambient light level using the **photoresistor (ADC)**.
 - Drive the **RGB LED (PWM)** (choose a warm color, like yellow/orange).
 - Map the values inversely: As the room gets darker (ADC value drops), the PWM duty cycle should increase so the LED gets brighter. If the room is fully lit, the LED should be completely off.
-

The LED Strip "Dial" Mapping Create a visual progress bar that perfectly mimics the physical rotation of the potentiometer knob.

- **Task:** Continuously read the value of the **potentiometer (ADC)**.
 - Map the total ADC range directly to the length of an LED strip using the 4 single-color LEDs (**LED_RED**, **LED_GREEN**, **LED_BLUE**, **LED_YELLOW**).
 - **The Logic:** Divide the potentiometer's rotation into 5 distinct zones:
 - **0% - 19%:** All LEDs OFF.
 - **20% - 39%:** **LED_RED** ON.
 - **40% - 59%:** **LED_RED** and **LED_GREEN** ON.
 - **60% - 79%:** **LED_RED**, **LED_GREEN**, and **LED_BLUE** ON.
 - **80% - 100%:** All 4 LEDs ON.
 - Update the LEDs continuously inside your loop. As you turn the knob clockwise, the "strip" of light should grow perfectly in sync with your hand.
-

Potentiometer Color-Temperature Dial Use the potentiometer to change the "temperature" of the RGB LED, simulating a smart lightbulb.

- Read the potentiometer ADC value.
 - Map the lowest ADC values to **Pure Blue** (Cold).
 - Map the middle ADC values to **White** (Red + Green + Blue mixed equally).
 - Map the highest ADC values to **Orange/Red** (Warm).
-

Auto-Calibrating Sensor Different rooms have different lighting, meaning a hardcoded photoresistor threshold won't always work. Write a program that calibrates itself!

- **Phase 1 (Calibration):** When the board boots, flash **LED_YELLOW** to indicate calibration mode. For 5 seconds, the user should cover the photoresistor completely, and then shine a light directly on it. Your code must track and save the minimum and maximum ADC values seen during this 5-second window.
 - **Phase 2 (Operation):** Turn off **LED_YELLOW** and turn on **LED_GREEN**. Now, map the current photoresistor reading to a PWM LED. Use the dynamically recorded min/max values as your 0% and 100% bounds instead of the theoretical 12/14-bit hardware limits.
-

The Hysteresis Nightlight Analog sensors are noisy, and simply turning an LED on/off at a single specific threshold will cause the LED to flicker rapidly when the light level is right at that boundary.

- Read the photoresistor (ADC).
- Implement **hysteresis** (two different thresholds):
 - If the ADC value goes *below* 30% of the maximum (it's getting dark), turn the **LED_BLUE** ON (or fade it up with PWM).
 - Do NOT turn the LED off until the ADC value goes *above* 50% of the maximum (it's definitely bright again).
- Between 30% and 50%, the LED should just maintain whatever state it is currently in.

Breathing LED with Adjustable Speed Create the classic "breathing" LED effect, but allow the user to control the speed of the breath dynamically.

- Write a loop that smoothly fades **LED_YELLOW** from 0% to 100% and back down to 0% using a loop of small duty cycle increments.
- Continuously read the potentiometer (ADC).
- Map the potentiometer's value to the delay duration inside your breathing loop (e.g., **Timer::after_millis(delay).await**).
- If the potentiometer is at 0, the breathing cycle should be very fast (e.g., 1 second total). If it is at maximum, the breathing cycle should be very slow (e.g., 5 seconds total).

The Full-Spectrum Color Wheel Instead of blending between just two colors, write a program that sweeps smoothly through the entire RGB color wheel using the potentiometer.

- Continuously read the **potentiometer** using the ADC.
- Divide the total ADC range into three equal mathematical zones.
- Map the ADC values in each zone to the appropriate PWM duty cycles for the **Red**, **Green**, and **Blue** channels to create a continuous crossfade:
 - **Zone 1 (0% to 33% ADC):** Fade from Pure Red (100% R) to Pure Green (100% G). Blue is OFF.
 - **Zone 2 (34% to 66% ADC):** Fade from Pure Green (100% G) to Pure Blue (100% B). Red is OFF.
 - **Zone 3 (67% to 100% ADC):** Fade from Pure Blue (100% B) back to Pure Red (100% R). Green is OFF.

The Visual Metronome Build a customizable metronome that flashes the RGB LED on the beat.

- **The Tempo:** Continuously read the potentiometer. Map its value to a BPM (Beats Per Minute) range of 60 to 240.
- **The Beat:** Flash the **Blue** channel of the RGB LED for exactly 50ms every time a beat occurs. The delay between beats is calculated from the BPM ($\text{Delay in ms} = 60000 / \text{BPM}$).
- **The Switch:** Use switch **S1** to act as a Play/Pause button. If paused, the LED stays completely off and ignores the timer, but you should still update the BPM variable in the background so it starts at the right speed when unpaused.

The Dashboard Dimmer (ADC + PWM + GPIO) Simulate the dashboard backlights of a car.

- Configure all 4 single-color LEDs (`LED_RED`, `LED_GREEN`, `LED_BLUE`, `LED_YELLOW`) to be driven by PWM.
 - Read the **potentiometer** and use it to set the master brightness (duty cycle) for all 4 LEDs simultaneously. As you turn the knob, all 4 LEDs should dim or brighten together.
 - **The Headlight Switch:** Use `S1` as an ON/OFF toggle switch. If the switch is OFF, force all LEDs to 0% brightness, ignoring the potentiometer. If the switch is ON, the LEDs instantly return to the brightness level currently dictated by the potentiometer.
-

The Single-Button RGB Color Mixer Mix your own custom color on the RGB LED, locking in one channel at a time.

- You will have three state variables: `locked_red`, `locked_green`, and `locked_blue`.
- Start in "Edit Red" mode. The potentiometer directly controls the Red PWM channel. Green and Blue are forced OFF.
- When you press `S1`, you transition to "Edit Green" mode. The Red channel stays frozen at whatever value the potentiometer was just at. Now, the potentiometer controls only the Green channel.
- Press `S1` again to freeze Green and edit Blue.
- Press `S1` a final time to freeze Blue. Now the RGB LED is displaying your fully mixed custom color, and turning the potentiometer does nothing until you press `S1` to reset and start over at Red.

SPI&I2C

The "Shake" Detector Create a system that detects if the board is being dropped or shaken violently.

- Initialize the MPU-6000 via SPI and set the Accelerometer Full Scale to **-/+ 16g**.
- Write a continuous loop that reads the X, Y, and Z accelerometer registers (**0x3B** through **0x40**).
- Calculate the total acceleration vector magnitude.
- *Note: Since 1g is Earth's baseline gravity, resting the board should yield a magnitude of roughly 1g.*
- If the total magnitude spikes above 2.5g or drops below 0.2g (freefall), instantly turn on **LED_RED** for 3 seconds to indicate a "Fault".

SPI Register Dump (Diagnostics Tool) Write a utility function to verify the MPU-6000's configuration.

- Instead of reading just one register, take advantage of the MPU-6000's sequential read capability.
- Send a single SPI transfer starting at the **ACCEL_CONFIG** register (**0x1C**), but make your receive buffer large enough to read the next 10 consecutive registers in one go.
- Iterate through the received buffer and print the hex value of each register to the console using `defmt::info!("Reg {}: {=u8:#X}", address, val)`.

The 24-bit Burst Read Barometer Configure the BMP390 **PWR_CTRL** for Normal mode with *both* Temperature and Pressure enabled.

- Perform a single I2C **write_read** starting at register **0x04** (**DATA_0**) with a receive buffer of 6 bytes to grab all pressure and temperature data at exactly the same time.
 - Reconstruct the 24-bit raw pressure value and the 24-bit raw temperature value using bitwise shifts (**<<**).
 - Print the raw 24-bit integer values to the console.
-

The Weather Station Dashboard Build a visual dashboard that displays the current environment data.

- Read the Temperature and Pressure from the BMP390 (I2C) every 2 seconds.
 - Apply the compensation formulas to get the actual float values, then format them into a `heapless::String` (e.g., "Temp: 24.5 C", "Press: 1013 hPa").
 - Use the `mipidsi` and `embedded_graphics` crates to draw this text onto the ST7735 screen (SPI).
-

The Digital Bubble Level Simulate a construction bubble level using the MPU-6000 and the 4 single-color LEDs.

- Continuously read the Accelerometer X and Y axes via SPI.
 - Define a "level" threshold (e.g., raw value close to 0).
 - Map the X-axis to `LED_RED` (tilt left) and `LED_GREEN` (tilt right).
 - Map the Y-axis to `LED_BLUE` (tilt forward) and `LED_YELLOW` (tilt backward).
 - If the board is tilted in a specific direction, turn on the corresponding LED. The further the tilt, the faster that LED should blink (using an async task or a mapped delay). If it is perfectly flat, all LEDs should be OFF.
-

SPI/I2C Protocol Bridge Write a program that uses an I2C device to dictate how you interact with an SPI device.

- **Task:** Use the BMP390 (I2C) temperature to control the SPI reporting rate of the MPU-6000.
- Read the temperature every second.
- If the temperature is below 25°C, read and print the MPU-6000 (SPI) gyroscope data every **1000ms**.
- If the temperature is above 25°C, read and print the MPU-6000 data every **200ms**.

The Digital Thermostat Build a classic thermostat interface using the temperature sensor and buttons.

- **Task:** Define a `target_temperature` variable in your code (e.g., 24.0 °C).
- Use `S1` to increase the target temperature by 0.5°C, and `S2` to decrease it by 0.5°C.
- Read the actual temperature from the BMP390 every 1 second.
- If the actual temperature is *lower* than the target, turn ON `LED_RED` (simulating a heater).
- If the actual temperature is *higher* than the target, turn ON `LED_BLUE` (simulating air conditioning).
- If it is within 0.5°C of the target, turn ON `LED_GREEN` (temperature is perfect).
- Print the target and actual temperatures to the console on every update.

SPI Screen Thermometer Bar Instead of just printing text to the screen, draw a dynamic visual graphic.

- **Task:** Read the BMP390 temperature every 500ms.
- Draw a vertical "thermometer bar" on the ST7735 screen.
- Map a temperature range (e.g., 20°C to 35°C) to the pixel height of the screen (0 to 160 pixels).
- If the temperature rises, draw a red filled rectangle growing upwards from the bottom of the screen.

The Manual I2C Address Sweeper Instead of a `for` loop that instantly scans the whole I2C bus, build a manual diagnostic tool.

- **Task:** Start a variable `current_address` at `0x08`.
- When you press `S1`, increment the address (wrap around back to `0x08` if it exceeds `0x77`).
- Every time the address changes, perform a 1-byte dummy `write_read` to that address.
- If the device ACKs (meaning a device exists at that address), turn `LED_GREEN` ON.
- If the device NACKs (no device present), turn `LED_RED` ON.

The LED Step Tracker

Build a simple step counter using the accelerometer, indicating your progress with the on-board LEDs.

- Read the **Z-axis** of the MPU-6000 Accelerometer continuously.
- Detect a "step" when the Z-axis acceleration spikes above **1.5g**. Once a step is detected, use `debounce` to pause execution before detecting another step.
- Maintain a `step_count` variable. Update the 4 single-color LEDs (`LED_RED`, `LED_GREEN`, `LED_BLUE`, `LED_YELLOW`) to act as a repeating progress bar based on your step count:
 - **1 Step:** `LED_RED` ON.
 - **2 Steps:** `LED_RED` and `LED_GREEN` ON.
 - **3 Steps:** `LED_RED`, `LED_GREEN`, and `LED_BLUE` ON.
 - **4 Steps:** All 4 LEDs ON.
 - **5 Steps:** All LEDs OFF (the cycle restarts).

Barometric Altitude Alarm

Use the pressure sensor to detect if the board is being carried up or down the stairs.

- **Task:** When the board boots, take 10 pressure readings from the BMP390 and calculate the average. Save this as your `baseline_pressure`.
 - **The Loop:** Read the pressure every 500ms.
 - **The Alarm:** If the current pressure drops by more than **30 Pascals** from the baseline (indicating you went *up* in altitude), turn ON `LED_BLUE`.
 - If the pressure rises by more than **30 Pascals** (indicating you went *down*), turn ON `LED_YELLOW`.
 - If the pressure is within +/- 30 Pascals of the baseline, keep all LEDs OFF.
-

The Physical Combination Lock

Instead of pushing buttons, unlock the device by physically tilting it in a secret sequence.

- **Task:** The secret sequence is: **Tilt Left, Tilt Right, Tilt Forward**.
 - **The Logic:** Read the MPU-6000 X and Y axes continuously.
 - Tilt Left: X-axis > 0.5g
 - Tilt Right: X-axis < -0.5g
 - Tilt Forward: Y-axis > 0.5g
 - Use a state machine to track the user's progress. If they complete the sequence in the exact right order, turn ON `LED_GREEN` and freeze the program.
 - If they make a wrong move (e.g., tilting forward when they were supposed to tilt right), flash `LED_RED` and force them to start over.
-

The 1-Byte Min/Max Tracker Instead of complex floating-point math, track the absolute highest and lowest temperatures using simple whole numbers and single bytes of memory.

- Reserve **2 bytes** in your EEPROM. Address `0x00` will hold the Maximum Temperature, and address `0x01` will hold the Minimum Temperature.
 - On startup, read the 1-byte Max from `0x00` and the 1-byte Min from `0x01`.
 - Read the BMP390 temperature every 2 seconds. Convert the floating-point temperature into a whole number by casting it.
 - Compare the new temperature to your saved Min and Max:
 - If `temp_int` is strictly greater than your saved Max, update your Max variable and write that **single byte** back to EEPROM address `0x00`.
 - If `temp_int` is strictly less than your saved Min, update your Min variable and write that **single byte** back to EEPROM address `0x01`.
 - Print the Current, Min, and Max temperatures to the console using `defmt::info!` on every loop.
-

Pressure-Based Screen Theme Change the UI's color palette based on environmental conditions.

- **Task:** Read the pressure from the BMP390.
 - Map the raw pressure value to a background color for the ST7735 screen.
 - For example: If the pressure is low (e.g., < 1000 hPa, indicating stormy/rainy weather), use `screen.clear(Rgb565::NAVY)`.
 - If the pressure is high (> 1015 hPa, indicating clear skies), use `screen.clear(Rgb565::CYAN)`.
 - Draw the actual pressure text in the center of the screen in White.
-

The "Stillness" Monitor Gyroscopes measure rotation. If the board is perfectly flat but vibrating or rotating, the accelerometer might not catch it, but the gyro will!

- **Task:** Continuously read the X, Y, and Z axes of the MPU-6000 **Gyroscope** (registers `0x43` to `0x48`).
 - Convert the raw bytes into three `i16` values.
 - If *any* of the three axes reads a value greater than `500` or less than `-500` (meaning the board is being twisted or turned), turn `LED_RED` ON and `LED_GREEN` OFF.
 - If all three axes are resting near `0` (between `-500` and `500`), turn `LED_GREEN` ON and `LED_RED` OFF.
-

The "Comfort Zone" Indicator Build a visual temperature gauge using the four colored LEDs.

- **Task:** Read the raw temperature from the BMP390 and convert it to Celsius.
 - If the temperature is **Below 20°C**: Turn ON `LED_BLUE` (Too cold).
 - If the temperature is **Between 20°C and 25°C**: Turn ON `LED_GREEN` (Perfect).
 - If the temperature is **Between 25°C and 30°C**: Turn ON `LED_YELLOW` (Getting warm).
 - If the temperature is **Above 30°C**: Turn ON `LED_RED` (Too hot).
-

Tilt-Activated Thermometer

Reading I2C sensors constantly can waste power. Build a system that only reads the temperature when you "ask" it to by tilting the board.

- **Task:** Poll the MPU-6000 Accelerometer X-axis every 100ms.
- Normally, the board does nothing.
- If you tilt the board sharply to the left (X-axis < -0.7g), trigger a single I2C read of the BMP390 temperature.
- Print the temperature to the console, and flash `LED_BLUE` once to confirm the reading was taken.

The EEPROM Boot Counter Display Combine persistent memory with the display so you can visually see how many times your code has run.

- **Task:** On boot, read the first two bytes (`0x0000` and `0x0001`) of the AT24C256 EEPROM. Combine them into a `u16`.
- If the value is `0xFFFF`, treat it as `0`.
- Add `1` to this number.
- Write the new number back to `0x0000` and `0x0001`.
- Format the number into a string (e.g., "`Boot Count: 5`") and draw it directly onto the ST7735 screen.

Digital Stopwatch Create a functioning stopwatch with Start, Stop, and Reset controls.

- **Task:** Maintain a `tenths_of_seconds` integer counter.
- **Controls:**
 - `S1` toggles the timer between **Running** and **Paused**.
 - `S2 Resets` the timer back to `0` (but only works if the timer is paused).
- **The Loop:** If the timer is Running, `Timer::after_millis(100).await`, then increment the counter.
- Format the counter into a string like "`Time: 12.4 s`" and draw it to the ST7735 screen.

Temperature Trend Indicator

Don't just measure the temperature—measure where it's going.

- **Task:** Read the BMP390 temperature every 2 seconds.
 - Maintain a `previous_temperature` variable.
 - Compare the current reading to the previous reading:
 - If the temperature has **risen** by more than `0.2` since the last reading, turn ON `LED_RED` (Warming up).
 - If the temperature has **fallen** by more than `0.2`, turn ON `LED_BLUE` (Cooling down).
 - If the temperature is **stable** (within `-/+ 0.2`), turn ON `LED_GREEN`.
 - Update `previous_temperature` to the new value at the end of the loop.
-